

ountcodec

varchar conversionratec
date creditholdc
varchar entityc
varchar excludefromeoastarterbronzeofferc
varchar parentc
varchar porequiredc
varchar portalnameurlc
varchar portalrequiredc
varchar sfdcsync
varchar sspchannelc
varchar supportholdcc

}

Subscription {

varchar-fk accountid
boolean autorenew
varchar-fk billtocontactid
varchar-fk billtocontactsnapshotid
date cancelleddate
decimal229 cmrr
date contractacceptancedate
date contracteffectivedate
varchar createdbyid
timestampwithtimezone createddate
varchar-fk creatoraccountid
varchar creatorinvoiceownerid
bigint currentterm
varchar currenttermperiodtype
varchar-pk id
bigint initialterm
varchar initialtermperiodtype
varchar-fk invoiceownerid
boolean isinvoiceseparate
varchar name
varchar notes
timestampwithtimezone originalcreateddate
varchar-fk originalid
varchar paymentterm
varchar-fk previoussubscriptionid
varchar-fk rampid
varchar renewalsetting
bigint renewalterm
varchar renewaltermperiodtype
varchar revision
date serviceactivationdate
varchar status
date subscriptionenddate
date subscriptionstartdate
varchar-fk subscriptionversionamendmentid
date termenddate

```

    date termstartdate
    varchar termtype
    varchar-fk updatedbyid
    timestampwithtimezone updateddate
    bigint version
    varchar cpqbundlejsonidqt
    date opportunityclosedateqt
    varchar opportunitynameqt
    varchar quotebusinessstypeqt
    varchar quotenumberqt
    varchar quotetypeqt
    varchar autorenewc
    varchar churncc
    varchar contractautorenewc
    varchar contractoperationalmetricsc
    varchar contractseatreconciliationc
    varchar endcustomerdetailsc
    varchar eoastarterbronzeofferacceptedc
    varchar excludefromrenewalreportcc
    varchar excludefromanalysisc
    varchar gitlabnamespaceidc
    varchar gitlabnamespacenamec
    varchar multiyeardealsubscriptionlinkagec
    varchar opportunityidc
    varchar purchaseorderc
    varchar recurlyidc
    varchar renewalsubscriptioncc
    varchar sfdcsync
    varchar turnonautorenewc
    varchar turnoncloudlicensingc
    varchar turnonoperationalmetricsc
    varchar turnonseatreconciliationc
}
Product{
    boolean allowfeaturechanges
    varchar category
    varchar-fk createdbyid
    timestampwithtimezone createddate
    varchar description
    date effectiveenddate
    date effectivestartdate
    varchar-pk id
    varchar name
    varchar sku
    varchar-fk updatedbyid
    timestampwithtimezone updateddate
}
ProductRatePlan{
    varchar-fk createdbyid
    timestampwithtimezone createddate

```

```

    varchar description
    date effectiveenddate
    date effectivestartdate
    varchar-pk id
    varchar name
    varchar-fk productid
    varchar-fk updatedbyid
    timestampwithtimezone updateddate
    varchar guidedsellingc
    varchar legacytieruidisplayc
    varchar prpcategoryc
}
ProductRatePlanCharge{
    varchar accountingcode
    varchar-fk accountreceivableaccountingcodeid
    varchar-fk adjustmentliabilityaccountingcodeid
    varchar-fk adjustmentrevenueaccountingcodeid
    varchar applydiscountto
    bigint billcycleday
    varchar billcycletype
    varchar billingperiod
    varchar billingperiodalignment
    varchar billingtiming
    varchar chargemodel
    varchar charge type
    varchar-fk contractassetaccountingcodeid
    varchar-fk contractliabilityaccountingcodeid
    varchar-fk contractrecognizedrevenueaccountingcodeid
    varchar-fk createdbyid
    timestampwithtimezone createddate
    decimal229 defaultquantity
    varchar deferredrevenueaccount
    varchar-fk deferredrevenueaccountingcodeid
    varchar description
    varchar-fk discountclassid
    varchar discountlevel
    varchar enddatecondition
    varchar-pk id
    decimal229 includedunits
    boolean legacyrevenuereporting
    varchar listpricebase
    decimal229 maxquantity
    decimal229 minquantity
    varchar name
    bigint numberofperiod
    varchar overagecalculationoption
    varchar overageunusedunitscreditooption
    varchar pricechangeoption
    decimal229 priceincreasepercentage
    varchar-fk productrateplanid

```

```

varchar ratinggroup
varchar recognizedrevenueaccount
varchar-fk recognizedrevenueaccountingcodeid
varchar revenuerecognitionrulename
varchar revreccode
varchar revrectriggercondition
varchar smoothingmodel
bigint specificbillingperiod
boolean taxable
varchar taxcode
varchar taxmode
varchar triggerevent
varchar-fk unbilledreceivablesaccountingcodeid
varchar uom
varchar-fk updatedbyid
timestampwithtimezone updateddate
bigint uptoperiods
varchar uptoperiodstype
varchar usagerecordratingoption
boolean usediscountspecificaccountingcode
boolean usetenantdefaultforpricechange
varchar weeklybillcycleday
varchar allocationeligibleflagc
varchar bundleidc
varchar pobstratifierc
varchar sspeligibleflagc
varchar trueuprevenueetermmmonthsc
}
ProductRatePlanChargeTier{
  boolean active
  varchar-fk createdbyid
  timestampwithtimezone createddate
  varchar currency
  decimal229 discountamount
  decimal229 discountpercentage
  decimal229 endingunit
  varchar-pk id
  decimal229 includedunits
  decimal229 overageprice
  decimal229 price
  varchar priceformat
  varchar-fk productrateplanchargeid
  decimal229 startingunit
  integer tier
  varchar-fk updatedbyid
  timestampwithtimezone updateddate
}
RatePlan {
  varchar-fk amendmentid
  varchar amendmenttype

```

```

varchar-fk createdbyid
timestampwithtimezone createddate
varchar-pk id
varchar name
varchar-fk productrateplanid
varchar-fk subscriptionid
varchar-fk updatedbyid
timestampwithtimezone updateddate
date synctriggerupdatec
}
RatePlanCharge {
  varchar accountingcode
  varchar-fk accountreceivableaccountingcodeid
  varchar applydiscountto
  bigint billcycleday
  varchar billcycletype
  varchar billingperiod
  varchar billingperiodalignment
  varchar billingtiming
  date chargedthroughdate
  varchar chargemodel
  varchar chargenumber
  varchar chargetype
  varchar-fk createdbyid
  timestampwithtimezone createddate
  varchar-fk deferredrevenueaccountingcodeid
  varchar description
  varchar discountlevel
  decimal229 dmrc
  decimal229 dtcv
  date effectiveenddate
  date effectivestartdate
  varchar enddatecondition
  varchar-pk id
  boolean islastsegment
  boolean isprocessed
  varchar listpricebase
  decimal229 mrr
  varchar name
  bigint numberofperiods
  varchar-fk originalid
  varchar overagecalculationoption
  varchar overageunusedunitscreditooption
  varchar pricechangeoption
  decimal229 priceincreasepercentage
  date processedthroughdate
  varchar-fk productrateplanchargeid
  decimal229 quantity
  varchar-fk rateplanid
  varchar ratinggroup

```

varchar-fk recognizedrevenueaccountingcodeid
varchar revenue recognitionrulename
varchar revreccode
varchar revrectriggercondition
integer segment
bigint specificbillingperiod
date specificenddate
decimal229 tcv
date triggerdate
varchar triggerevent
varchar uom
varchar-fk updatedbyid
timestampwithtimezone updateddate
bigint uptoperiods
varchar uptoperiodstype
bigint version
varchar weeklybillcycleday
varchar psbundleidc
varchar rampdealidc
date synctriggerupdatec
varchar unbilledarflagc
}

RatePlanChargeTier {
 varchar-fk createdbyid
 timestampwithtimezone createddate
 varchar currency
 decimal229 discountamount
 decimal229 discountpercentage
 decimal229 endingunit
 varchar-pk id
 decimal229 includedunits
 decimal229 overageprice
 decimal229 price
 varchar priceformat
 varchar-fk rateplanchargeid
 decimal229 startingunit
 bigint tier
 varchar-fk updatedbyid
 timestampwithtimezone updateddate
}

Amendment {
 boolean autorenew
 varchar code
 date contracteffectivedate
 varchar-fk createdbyid
 timestampwithtimezone createddate
 bigint currentterm
 varchar currenttermperiodtype
 date customeracceptancedate
 varchar description

```

    date effectivedate
    varchar-pk id
    varchar name
    varchar renewalsetting
    bigint renewalterm
    varchar renewaltermperiodtype
    date resumedate
    date serviceactivationdate
    date specificupdateddate
    varchar status
    varchar-fk subscriptionid
    date suspenddate
    date termstartdate
    varchar termtype
    varchar type
    varchar-fk updatedbyid
    timestampwithtimezone updateddate
}
Contact {
    varchar-fk accountid
    varchar address1
    varchar address2
    varchar city
    varchar country
    varchar county
    varchar-fk createdbyid
    timestampwithtimezone createddate
    varchar description
    varchar fax
    varchar firstname
    varchar homephone
    varchar-pk id
    varchar lastname
    varchar mobilephone
    varchar nickname
    varchar otherphone
    varchar otherphonetype
    varchar personalemail
    varchar postalcode
    varchar state
    varchar taxregion
    varchar-fk updatedbyid
    timestampwithtimezone updateddate
    varchar workemail
    varchar workphone
    varchar financeverifiedcontactc
}
PaymentMethod {
    varchar-fk accountid
    varchar achabacode

```

varchar achaccountname
varchar achaccountnumbermask
varchar achaccounttype
varchar achaddress1
varchar achaddress2
varchar achbankname
varchar achcity
varchar achcountry
varchar achpostalcode
varchar achstate
boolean active
varchar bankbranchcode
varchar bankcheckdigit
varchar bankcity
varchar bankcode
varchar bankidentificationnumber
varchar bankname
varchar bankpostalcode
varchar bankstreetname
varchar bankstreetnumber
varchar banktransferaccountname
varchar banktransferaccountnumbermask
varchar banktransferaccounttype
varchar banktransfertype
varchar businessidentificationcode
varchar city
varchar companyname
varchar country
varchar-fk createdbyid
timestampwithtimezone createddate
varchar creditcardaddress1
varchar creditcardaddress2
varchar creditcardcity
varchar creditcardcountry
integer creditcardexpirationmonth
integer creditcardexpirationyear
varchar creditcardholdername
varchar creditcardmasknumber
varchar creditcardpostalcode
varchar creditcardstate
varchar creditcardtype
varchar devicesessionid
varchar email
varchar existingmandate
varchar firstname
varchar iban
varchar-pk id
varchar identitynumber
varchar ipaddress
boolean iscompany


```
boolean issystem
date lastfailedsaletransactiondate
varchar lastname
timestampwithtimezone lasttransactiondatetime
varchar lasttransactionstatus
date mandatecreationdate
varchar mandateid
varchar mandatereason
varchar mandatereceived
varchar mandatestatus
date mandateupdatedate
smallint maxconsecutivepaymentfailures
varchar name
integer numconsecutivefailures
varchar paymentmethodstatus
smallint paymentretrywindow
varchar paypalbaid
varchar paypalemail
varchar paypalpreapprovalkey
varchar paypaltype
varchar phone
varchar postalcode
varchar secondtokenid
varchar state
varchar streetname
varchar streetnumber
varchar tokenid
integer totalnumberoferrorpayments
integer totalnumberofprocessedpayments
varchar type
varchar-fk updatedbyid
timestampwithtimezone updateddate
boolean usedefaultretryrule
}
Invoice {
  varchar-fk accountid
  decimal229 adjustmentamount
  decimal229 amount
  decimal229 amountwithouttax
  boolean autopay
  decimal229 balance
  varchar-fk billtocontactsnapshotid
  varchar comments
  varchar createdbyid
  timestampwithtimezone createddate
  decimal229 creditbalanceadjustmentamount
  date duedate
  varchar-pk id
  boolean includesonetime
  boolean includesrecurring
```

```

boolean includesusage
date invoicedate
varchar invoicenumbr
timestampwithtimezone lastemailsentdate
decimal229 paymentamount
varchar postedby
timestampwithtimezone posteddate
decimal229 refundamount
boolean reversed
varchar soldtocontactsnapshotid
varchar source
varchar-fk sourceid
varchar status
date targetdate
decimal229 taxamount
decimal229 taxexemptamount
varchar taxmessage
varchar taxstatus
varchar transferredtoaccounting
varchar-fk updatedbyid
timestampwithtimezone updateddate
varchar vatlocalcurrencyamountc
}
InvoiceItem {
  varchar-fk accountid
  varchar accountingcode
  varchar-fk accountreceivableaccountingcodeid
  varchar-fk adjustmentliabilityaccountingcodeid
  varchar-fk adjustmentrevenueaccountingcodeid
  varchar-fk amendmentid
  varchar-fk appliedtoinvoiceitemid
  decimal229 balance
  varchar-fk billtocontactid
  varchar-fk billtocontactsnapshotid
  decimal229 chargeamount
  timestampwithtimezone chargedate
  varchar chargename
  varchar-fk contractassetaccountingcodeid
  varchar-fk contractliabilityaccountingcodeid
  varchar-fk contractrecognizedrevenueaccountingcodeid
  varchar-fk createdbyid
  timestampwithtimezone createddate
  varchar-fk defaultpaymentmethodid
  varchar-fk deferredrevenueaccountingcodeid
  varchar description
  varchar-pk id
  varchar-fk invoiceid
  varchar-fk journalentryid
  varchar-fk parentaccountid
  varchar processingtype

```

varchar-fk productid
varchar-fk productrateplanchargeid
varchar-fk productrateplanid
decimal229 quantity
varchar-fk rateplanchargeid
varchar-fk rateplanid
varchar-fk recognizedrevenueaccountingcodeid
date revrecstartdate
date serviceenddate
date servicestartdate
varchar sku
varchar-fk soldtocontactid
varchar-fk soldtocontactsnapshotid
varchar-fk subscriptionid
decimal229 taxamount
varchar taxcode
decimal229 taxexemptamount
varchar taxmode
varchar-fk unbilledreceivablesaccountingcodeid
decimal229 unitprice
varchar uom
varchar-fk updatedbyid
timestampwithtimezone updateddate

}

TaxationItem{
 varchar accountingcode
 varchar-fk accountreceivableaccountingcodeid
 decimal229 balance
 varchar-fk createdbyid
 timestampwithtimezone createddate
 decimal229 creditamount
 decimal229 exemptamount
 varchar-pk id
 varchar-fk invoiceitemid
 varchar-fk journalentryid
 varchar jurisdiction
 varchar locationcode
 varchar name
 decimal229 paymentamount
 varchar-fk salestaxpayableaccountingcodeid
 varchar-fk taxableitemsnapshotid
 decimal229 taxamount
 varchar taxcode
 varchar taxcodedescription
 date taxdate
 varchar taxmode
 decimal229 taxrate
 varchar taxratedescription
 varchar taxratetype
 varchar-fk updatedbyid

```
timestampwithtimezone updateddate
}
PaymentMethodSnapshot {
  varchar-fk accountid
  varchar achabacode
  varchar achaccountname
  varchar achaccountnumbermask
  varchar achaccounttype
  varchar achbankname
  varchar bankbranchcode
  varchar bankcheckdigit
  varchar bankcity
  varchar bankcode
  varchar bankidentificationnumber
  varchar bankname
  varchar bankpostalcode
  varchar bankstreetname
  varchar bankstreetnumber
  varchar banktransferaccountname
  varchar banktransferaccountnumbermask
  varchar banktransferaccounttype
  varchar banktransfertype
  varchar businessidentificationcode
  varchar city
  varchar companyname
  varchar country
  timestampwithtimezone createddate
  varchar creditcardaddress1
  varchar creditcardaddress2
  varchar creditcardcity
  varchar creditcardcountry
  integer creditcardexpirationmonth
  integer creditcardexpirationyear
  varchar creditcardholdername
  varchar creditcardmasknumber
  varchar creditcardpostalcode
  varchar creditcardstate
  varchar creditcardtype
  varchar devicesessionid
  varchar email
  varchar existingmandate
  varchar firstname
  varchar iban
  varchar-pk id
  varchar identitynumber
  varchar ipaddress
  boolean iscompany
  date lastfailedsaletransactiondate
  varchar lastname
  timestampwithtimezone lasttransactiondatetime
```

```

varchar lasttransactionstatus
date mandatecreationdate
varchar mandateid
varchar mandatereason
varchar mandatereceived
varchar mandatestatus
date mandateupdatedate
smallint maxconsecutivepaymentfailures
varchar name
integer numconsecutivefailures
varchar paymentmethodid
varchar paymentmethodstatus
smallint paymentretrywindow
varchar paypalbaid
varchar paypalemail
varchar paypalpreapprovalkey
varchar paypaltype
varchar phone
varchar postalcode
varchar secondtokenid
varchar state
varchar streetname
varchar streetnumber
varchar tokenid
integer totalnumberoferrorpayments
integer totalnumberofprocessedpayments
varchar type
timestampwithtimezone updateddate
boolean usedefaultretryrule
}
InvoicePayment {
    varchar-fk accountreceivableaccountingcodeid
    decimal229 amount
    varchar-fk cashaccountingcodeid
    varchar-fk createdbyid
    timestampwithtimezone createddate
    varchar-pk id
    varchar-fk invoiceid
    varchar-fk journalentryid
    varchar-fk paymentid
    decimal229 refundamount
    varchar-fk updatedbyid
    timestampwithtimezone updateddate
}
Payment {
    varchar-fk accountid
    varchar accountingcode
    decimal229 amount
    decimal229 appliedamount
    decimal229 appliedcreditbalanceamount

```

varchar authtransactionid
varchar bankidentificationnumber
timestampwithtimezone cancelledon
varchar comment
varchar-fk createdbyid
timestampwithtimezone createddate
varchar currency
date effectivedate
varchar gateway
varchar gatewayorderid
varchar gatewayreconciliationreason
varchar gatewayreconciliationstatus
varchar gatewayresponse
varchar gatewayresponsecode
varchar gatewaystate
varchar-pk id
boolean isstandalone
timestampwithtimezone markedforsubmissionon
varchar paymentmethodid
varchar-fk paymentmethodsnapshotid
varchar paymentnumber
varchar payoutid
varchar referencedpaymentid
varchar referenceid
decimal229 refundamount
varchar secondpaymentreferenceid
timestampwithtimezone settledon
varchar softdescriptor
varchar softdescriptorphone
varchar source
varchar sourcename
varchar status
timestampwithtimezone submittedon
varchar transactionsource
varchar transferredtoaccounting
varchar type
decimal229 unappliedamount
varchar updatedbyid
timestampwithtimezone updateddate
}

Refund {
 varchar-fk accountid
 varchar accountingcode
 decimal229 amount
 varchar associatedtransactionnumber
 timestampwithtimezone cancelledon
 varchar comment
 varchar-fk createdbyid
 timestampwithtimezone createddate
 varchar gateway

```

varchar gatewayreconciliationreason
varchar gatewayreconciliationstatus
varchar gatewayresponse
varchar gatewayresponsecode
varchar gatewaystate
varchar-pk id
timestampwithtimezone markedforsubmissionon
varchar methodtype
varchar-fk paymentmethodid
varchar-fk paymentmethodsnapshotid
varchar payoutid
varchar reasoncode
varchar referenceid
date refunddate
varchar refundnumber
timestampwithtimezone refundtransactiontime
varchar secondrefundreferenceid
timestampwithtimezone settledon
varchar softdescriptor
varchar softdescriptorphone
varchar sourcetype
varchar status
timestampwithtimezone submittedon
varchar transferredtoaccounting
varchar type
varchar-fk updatedbyid
timestampwithtimezone updateddate
}
RefundInvoicePayment {
  varchar-fk accountreceivableaccountingcodeid
  varchar-fk cashaccountingcodeid
  varchar-fk createdbyid
  timestampwithtimezone createddate
  varchar-pk id
  varchar-fk invoicepaymentid
  varchar-fk journalentryid
  decimal229 refundamount
  varchar-fk refundid
  varchar-fk updatedbyid
  timestampwithtimezone updateddate
}

```

CustomersDot (Customer Portal)

```

mermaid
erDiagram
  cloudactivations {
    bigint id
    timestampwithouttimezone createdat
  }

```

```

timestampwithouttimezone updatedat
bigint customerid
charactervarying activationcode
charactervarying subscriptionname
boolean supersonicsaware
}

coupons {
  bigint id
  charactervarying batchid
  charactervarying code
  smallint status
}

customers {
  integer id
  charactervarying firstname
  charactervarying lastname
  timestampwithouttimezone createdat
  timestampwithouttimezone updatedat
  charactervarying email
  charactervarying encryptedpassword
  charactervarying resetpasswordtoken
  timestampwithouttimezone resetpasswordsentat
  timestampwithouttimezone remembercreatedat
  integer signincount
  timestampwithouttimezone currentsigninat
  timestampwithouttimezone lastsigninat
  inet currentsigninip
  inet lastsigninip
  charactervarying provider
  charactervarying uid
  charactervarying zuoraaccountid
  charactervarying country
  charactervarying state
  charactervarying city
  charactervarying zipcode
  charactervarying vatcode
  charactervarying company
  charactervarying salesforceaccountid
  boolean billable
  charactervarying accesstoken
  charactervarying confirmationtoken
  timestampwithouttimezone confirmedat
  timestampwithouttimezone confirmationsentat
  charactervarying unconfirmedemail
  charactervarying address1
  charactervarying address2
  charactervarying companysize
  charactervarying authenticationtoken

```



```
    character_varying phonenumber
    boolean loginactivated
}
```

```
instancereviewsettings {
    integer id
    integer userscountthreshold
    timestampwithouttimezone createdat
    timestampwithouttimezone updatedat
}
```

```
instancereviews {
    integer id
    character_varying email
    character_varying firstname
    character_varying lastname
    character_varying company
    character_varying address
    character_varying phone
    character_varying version
    integer userscount
    integer groupscount
    integer projectscount
    integer issuescount
    integer mergerequestscount
    integer internalpipelinescount
    integer externalpipelinescount
    integer labelscount
    integer milestonescount
    integer notescount
    integer snippetscount
    character_varying country
    character_varying state
    character_varying employeerange
    boolean communicationoptedin
}
```

```
admins {
    integer id
    character_varying email
    character_varying encryptedpassword
    character_varying resetpasswordtoken
    timestampwithouttimezone resetpasswordsentat
    timestampwithouttimezone remembercreatedat
    integer signincount
    timestampwithouttimezone currentsigninat
    timestampwithouttimezone lastsigninat
    inet currentsigninip
    inet lastsigninip
    timestampwithouttimezone createdat
}
```

```
timestampwithouttimezone updatedat
character_varying authenticationtoken
boolean manageadmin
character_varying provider
character_varying uid
smallint customeraccesslevel
smallint licenseaccesslevel
}
```

```
licenses {
  bigint id
  uuid licensefilemd5
  bigint customerid
  bigint creatorid
  timestampwithtimezone createdat
  timestampwithtimezone updatedat
  timestampwithtimezone lastsyncdat
  timestampwithtimezone nextsyncat
  integer userscount
  integer previoususerscount
  integer trueupquantity
  date expiresat
  date startsat
  date trueupfrom
  date trueupto
  boolean trial
  boolean cloudlicensingenabled
  character_varying plancode
  character_varying planname
  character_varying zuorasubscriptionid
  character_varying email
  character_varying name
  character_varying company
  character_varying zuorasubscriptionname
  text notes
  text licensefile
  timestampwithtimezone activatedat
  boolean autorenewenabled
  boolean seatreconciliationenabled
  boolean operationalmetricsenabled
  boolean reconciliationcompleted
  boolean offlinecloudlicensingenabled
}
```

```
localrecords {
  bigint id
  timestampwithouttimezone createdat
  timestampwithouttimezone updatedat
  character_varying name
  text csv
}
```

}

```
orders {  
  integer id  
  integer customerid  
  charactervarying productrateplanid  
  charactervarying subscriptionid  
  charactervarying subscriptionname  
  date startdate  
  date enddate  
  integer quantity  
  timestampwithouttimezone createdat  
  timestampwithouttimezone updatedat  
  charactervarying glnamespaceid  
  charactervarying glnamespacename  
  charactervarying amendmenttype  
  boolean trial  
  timestampwithouttimezone lastextraciminutessyncat  
  charactervarying zuoraaccountid  
  timestampwithouttimezone increasedbillingratenotifiedat  
  boolean reconciliationaccepted  
  timestampwithouttimezone billingrateadjustedat  
  charactervarying billingratelastaction  
  smallint trialextensiontype  
  charactervarying source  
}
```

```
trialhistories {  
  integer id  
  charactervarying glnamespaceid  
  date startdate  
  date expiredon  
  timestampwithouttimezone createdat  
  timestampwithouttimezone updatedat  
  charactervarying glmsource  
  charactervarying glmcontent  
  charactervarying trialentity  
}
```

```
zuorauserwithhosts {  
  integer id  
  charactervarying zuorauserid  
  charactervarying host  
  timestampwithouttimezone createdat  
  timestampwithouttimezone updatedat  
}
```

```
selfmanagedinstanceactivations {  
  bigint id  
  timestampwithouttimezone createdat
```

```
    timestampwithouttimezone updatedat
    timestampwithouttimezone activatedat
    bigint selfmanagedinstanceid
    bigint cloudactivationid
}
```

```
selfmanagedinstances {
  bigint id
  timestampwithouttimezone createdat
  timestampwithouttimezone updatedat
  charactervarying instanceidentifier
}
```

```
licenseseatlinks {
  bigint id
  timestampwithouttimezone createdat
  timestampwithouttimezone updatedat
  bigint orderid
  integer activeusercount
  integer maxhistoricalusercount
  integer licenseusercount
  date licenseseatstartson
  charactervarying zuorasubscriptionname
  charactervarying zuorasubscriptionid
  timestampwithtimezone reporttimestamp
  bigint licenseid
  bigint selfmanagedinstanceactivationid
  charactervarying gitlabversion
  charactervarying hostname
  charactervarying instanceidentifier
  uuid licensemd5
}
```

```
reconciliations {
  bigint id
  integer orderid
  integer usercount
  timestampwithouttimezone reconciledoneat
  timestampwithouttimezone createdat
  timestampwithouttimezone updatedat
  text errormessage
  date reconcileon
  smallint skipreason
  timestampwithtimezone upcomingemailsentsat
  timestampwithtimezone completedemailsentsat
  numeric quotedinvoiceamount
  timestampwithtimezone quotedinvoiceat
}
```

```
leads {
```

```

    bigint id
    timestampwithtimezone createdat
    timestampwithtimezone updatedat
    date trialstartdate
    bigint namespaceid
    bigint userid
    boolean optin
    boolean currentlyintrial
    boolean isforbusinessuse
    charactervarying firstname
    charactervarying lastname
    charactervarying email
    charactervarying phone
    charactervarying companyname
    charactervarying employeesbucket
    charactervarying country
    charactervarying state
    charactervarying productinteraction
    charactervarying provider
    text commentcapture
    charactervarying glmcontent
    charactervarying glmsource
    timestampwithtimezone sentat
}

cloudactivations }o--|| customers : ""
selfmanagedinstanceactivations }o--|| cloudactivations : ""
licenses }o--|| customers : ""
orders }o--|| customers : ""
licenses }o--|| admins : ""
licenseseatlinks }o--|| licenses : ""
licenseseatlinks }o--|| orders : ""
reconciliations }o--|| orders : ""
licenseseatlinks }o--|| selfmanagedinstanceactivations : ""
selfmanagedinstanceactivations }o--|| selfmanagedinstances : ""

```

GitLab

```

mermaid
erDiagram
    ciminutesadditionalpacks {
        bigint id
        timestampwithtimezone createdat
        timestampwithtimezone updatedat
        bigint namespaceid
        date expiresat
        integer numberofminutes
        text purchasexid
    }

```

```
gitlabsubscriptions {
  bigint id
  timestampwithtimezone createdat
  timestampwithtimezone updatedat
  date startdate
  date enddate
  date trialendson
  integer namespaceid
  integer hostedplanid
  integer maxseatsused
  integer seats
  boolean trial
  date trialstartson
  boolean autorenew
  integer seatsinuse
  integer seatsowed
  smallint trialextensiontype
}

historicaldata {
  integer id
  date date
  integer activeusercount
  timestampwithouttimezone createdat
  timestampwithouttimezone updatedat
  timestampwithtimezone recordedat
}

licenses {
  integer id
  text data
  timestampwithouttimezone createdat
  timestampwithouttimezone updatedat
  boolean cloud
  timestampwithtimezone lastsyncedat
}

members {
  integer id
  integer accesslevel
  integer sourceid
  charactervarying sourcetype
  integer userid
  integer notificationlevel
  charactervarying type
  timestampwithouttimezone createdat
  timestampwithouttimezone updatedat
  bigint membernamespaceid
}
```

```

namespacelimits {
    bigint additionalpurchasedstoragesize
    date additionalpurchasedstorageendson
    integer namespaceid
    date temporarystorageincreaseendson
}

namespaces {
    integer id
    charactervarying name
    charactervarying path
    integer ownerid
    timestampwithouttimezone createdat
    timestampwithouttimezone updatedat
    charactervarying type
    charactervarying description
    charactervarying avatar
    integer parentid
    integer sharedrunnersminuteslimit
    bigint repositorysizelimit
    integer extrasharedrunnersminuteslimit
    timestampwithtimezone lastciminutesnotificationat
    integer lastciminutesusagenotificationlevel
    boolean sharedrunnersenabled
}

personalaccesstokens {
    integer id
    integer userid
    charactervarying name
    boolean revoked
    date expiresat
    timestampwithouttimezone createdat
    timestampwithouttimezone updatedat
    charactervarying scopes
    boolean impersonation
    charactervarying tokendigest
    boolean expirenotificationdelivered
    timestampwithtimezone lastusedat
    boolean afterexpirynotificationdelivered
}

planlimits {
    bigint id
    bigint planid
    integer storagesizelimit
}

plans {
    integer id

```

```

    timestampwithouttimezone createdat
    timestampwithouttimezone updatedat
    charactervarying name
    charactervarying title
}

upcomingreconciliations {
    bigint id
    bigint namespaceid
    date nextreconciliationdate
    date displayalertfrom
    timestampwithtimezone createdat
    timestampwithtimezone updatedat
}

usercreditcardvalidations {
    bigint userid
    timestampwithtimezone creditcardvalidatedat
    date expirationdate
    smallint lastdigits
    text holdername
    text network
}

users {
    integer id
    charactervarying email
    charactervarying name
    boolean admin
    integer projectslimit
    charactervarying username
    charactervarying state
    charactervarying firstname
    charactervarying lastname
    smallint usertype
}

gitlabsubscriptions |o--|| namespaces : ""
gitlabsubscriptions }o--|| plans : ""
members }o--|| namespaces : ""
members }o--|| users : ""
namespacelimits |o--|| namespaces : ""
namespaces }o--|| namespaces : ""
upcomingreconciliations }o--|| namespaces : ""
users }o--|| namespaces : ""
personalaccesstokens }o--|| users : ""
planlimits }o--|| plans : ""
usercreditcardvalidations |o--|| users : ""

```

SECTION: business-technology/enterprise-applications/entapps-crm/quote-to-cash/troubleshooting/index.md

Important Links Internal

- L&R Notice Board
- Need Access to CustomersDot?
 - to get a login to the CustomersDot, by opening an access request; remove all systems in the list except for the CustomersDot row.
- Make sure to save these credentials as distinct from your regular GitLab credentials.
- Need Access to the Version Application?
 - to get a login to the version application, open an access request and remove all systems in the list except for the row for GitLab DEV which you will use to OAuth into the application.
- Make sure to save these credentials as distinct from your regular GitLab credentials.
- Kyla's Quote Creation Videos
- Support: Common Licensing, Transaction and Billing Troubleshooting
- Sales Enablement Renewal Doc
- How to create a quote
 - new subscription quote
 - amend subscription quote
 - renew subscription quote
- Quote Approval instruction
- Good resource of helpful links from CSMs
- How GitLab.com Subscriptions Work

Important Links for Customers and Prospects

- Resources for Prospects who are trialing and need direction
 - Get started with GitLab
 - GitLab epics
- Instructions on how to renew self-managed
- Customer Facing: Licensing, Renewing, and Subscription FAQ
- Subscription Setup and Management (user facing)

True-Ups, Adding Seats, Users

User Counts: True Ups, Add-ons, Users Over License

What is a True Up?

A true-up is a back payment for a customer going over their license count during the previous year.

We use the true-up model so that the license never becomes a barrier to adoption for the client.

What causes a true-up error?

At renewal, it is possible that our customers have miscalculated the number of true-up users required for their license.

The true-up must match the number of users over license data collected on their admin dashboard or else the license will fail to activate.

Example:

Customer purchased 10 users at the start of their subscription; during the year of their subscription, they had 12 total users in license, adding 2 additional.

Their admin dashboard would read 2 users over license.

At the time of their renewal, they will be responsible for paying for those 2 users over license (as true-up users), along with the number of total number of users they want in the renewal subscription.

If the true-up is not added or has the incorrect number of users over the license, the license key will not work.

The customer is always free to renew for as many users as they would like, even if this number is lower than the previous year; the true-up amount just needs to be paid for in full.

If a customer is having issues obtaining the number of licenses needed to purchase, you can request a screenshot from them or else access their usage ping data through the version application.

If there is no data available through usage ping, we can conclude that the usage ping has been disabled in the instance, and you will need to obtain this information manually.

Customer Question: Why is there a discrepancy between "Maximum Users" and "Users" on the admin dashboard? (self-managed)

Everyday during the subscription, the instance records the Active User count for the day.

The following users do not count as billable Active Users:

- Guest users on Ultimate plans
- GitLab generate bots like Support Bot and Alert Bot
- Ghost users
- Blocked users
- Deactivated users

The "Maximum Users" count in the instance refers to this running list of active user counts and displays the maximum value.

Customer Question: How do I purchase more user seats for a self-managed instance of GitLab?

Communicate to customer

>>>

- Navigate to the CustomersDot and sign in using your GitLab.com account or request a one-time sign-in link
- Go to Manage Purchases
- Locate the subscription you wish to add more users to and click the button Add more seats
- Enter the additional amount of users you wish to have (i.e. you have 50 but want to have 60, so you have to enter 10)

- Click Proceed to checkout and finalize the purchase
- You will receive amended license key via email afterward

>>>

Licenses

What is a license?

- A license is only generated for self-managed customers or trials - SaaS does not utilize a license.
- The license is generated by our CustomersDot application and the customer will be emailed a license key. The license will also be available in the CustomersDot application.
- It is possible to generate license keys manually.
- If you need access to the license or the version applications, open an access request and remove all systems in the list except for the row GitLab DEV (required for license and version access).

You will use the dev credentials to OAuth into the license and version apps using login with GitLab so make sure to save these credentials in a way to differentiate them from your other GitLab credentials.

How do I know a license was sent?

When a license is sent, whether automated or manually, an activity is recorded in SFDC on the contact record.

How to ask the customer for a screenshot needed for users over license with Self-managed

Communicate to the customer

>>>

To help me provide better support for your renewal I'll need some more information, could you please provide a screenshot following the below instructions?

1. Navigate to Admin Area
1. Click License as it's described here and send it to me
 - For GitLab.com
1. Navigate to group settings
1. Navigate to billing
 - If you can share the details found there under Maximum Users and Users over License, then I will be able to send a quote over to you.

>>>

How can I ask Support to resend a license?

Open an issue using the License Issue template.

How do I change the license owner for self-managed instances with license.gitlab

Two options:

Support Request

Open an issue and request Support to do this.

Self Service: CustomersDot

1. Navigate to the CustomersDot admin dashboard
2. Select 'Licenses' from the side menu on the left
3. Find the company account through the search option
4. Select their most recent license that needs to be sent
5. Scroll down to the bottom and select duplicate license
6. After duplicate license change the email address

When might a license be manually generated?

Licenses only apply to self managed.

- In order to fix a problem/error: This would generally occur around a renewal after a customer tried to use the Web Store and entered in users counts that didn't match what the system expected.
- In order for a customer to trial a different tier than Ultimate.
- When a reseller has different discount rates for renewals and add-ons.
For this scenario, an order will be created with two lines of the same product but if a license gets automatically created in such case it will only be pulled for the first line of product and won't be complete so this requires manual license creation.
- Licenses are sometimes generated manually when a customer has add-ons for multi-year deals.

Manually Generate License for a Trial or to Amend a True Up Problem

Note: There are no license keys provided for GitLab.com subscriptions

Please open an issue and request Support to do this.

For Trial Licenses:

1. Navigate to the CustomersDot admin dashboard
2. Select 'Licenses' from the side menu
3. Type in customer name (exactly as it is in Salesforce) or use the domain name in their email address
4. When the license comes up, there might be one or there might be multiple
 - Navigate to the most recent license based on expires at date
 - Click on the Name
 - Scroll down to duplicate license
 - Type in the correct number of users
 - Check the box indicating this is a trial
 - Select the plan: make sure the starts at and expires at reflect the accurate dates for the trial because we cannot retract a license if you accidentally make the expiration date for a year from now they'll have free access for a year
 - Add notes
 - Click Create License

5. The license will automatically be sent to the email you enter, and you can also download the license from that screen and send as an attachment to prospect directly.

Generate License to Amend a True Up Problem (not for trial)

1. Navigate to the CustomersDot admin dashboard
2. Select 'Licenses' from the side menu
3. Type in customer name (exactly as it is in Salesforce) or use the domain name in their email address
4. When the license comes up, there might be one or there might be multiple
 - Navigate to the most recent license based on expires at date
 - Click on the Name
 - Scroll down to duplicate license
 - Type in the correct number of users or true-ups
 - Make sure the starts at and expires at dates are not changed
 - Add notes to give reference as to why you're amending the license along with the name of the manager who approved the true-up adjustment
 - Click Create License
5. The license will automatically be sent to the email on file, and you can also download the license from that screen and send as an attachment to prospect directly.

Terms (EULAs/TOS/T&Cs)

Overview

End User License Agreement (EULA), Terms of Service (TOS), or Term & Conditions (T&Cs) will be referred to generally as "Terms" in this documentation. Terms are applicable to all customers and are required to be accepted before using GitLab (self-managed or .com) even if it's a trial version.

When are Terms required to be acknowledged?

- Customers who purchase through the web store are required to acknowledge Terms during the process of purchasing.
- Customers who purchase from Sales accept Terms when they sign the order form.
- Users who want to trial a product.
- Customers who want to upload license to an instance.

What about EULA problems?

- Open an issue for EULA problems
- Open an issue for license problems

Who accepts the EULA?

The Terms are accepted upon installing the license into the Self-managed instance.

What is the difference between Terms (EULA/T&Cs/TOS) and a License?

- A EULA (End User License Agreement) or T&Cs (Terms and Conditions) are the legal agreements

for the terms of use and privacy policy that customers and users must agree to before using our software.

You can read through them on our company website.

- A license is a string of characters that is tied to the customer's subscription or trial that has a start and end date.

Without applying the license, the self managed software won't work as expected.

It is sent to the customer via email or available for download after Terms have been accepted.

How to check if a EULA has been sent?

EULAs are no longer sent to the end user.

The Terms are accepted upon installing the license into the Self-managed instance.

Trials

How do I request a trial for SaaS customer?

Submit an issue using the Plan Change Request template.

How long do trials last? Where can I see the end date?

Thirty days - you can see the end date if a namespace is on a trial in the CustomersDot admin.

How do I request to extend a SaaS trial for a customer?

For trial extensions open an issue.

How do I request a tier other than Gold or Ultimate for a customer?

1. open an issue using the Plan Change Request template.

- NOTE: It's not yet possible for users or GitLab employees to directly start a Bronze or Silver trial on a namespace.

- If one is needed, have the user initiate a Gold trial and then open an issue per the above link using the Plan Change Request template to have the plan manually changed. Implementation of this feature is being discussed in this epic.

Customer Question: how do I setup a SaaS trial?

Communicate to customer

>>>

- Have you created a group?

- If not, please do so; this documentation will help:

<<https://docs.gitlab.com/ee/user/group/create-a-new-group>>

- When you've created a group try the trial again using the email associated with that group here: <<https://customers.gitlab.com/trials/new?glcom=true>>

- If you still run into an error please let me know and I'll have support manually upgrade your trial group.

>>>

Trial users who need help

- Trial users are not entitled to access support unless they are sales assisted
- If a prospect working with Sales needs help they should email trials@gitlab.zendesk.com and there is no SLA

Subscription or Plan Problems

My customer is unable to apply a purchased subscription to their GitLab.com group

Ask the user to open a support ticket

I want to change the plan of a GitLab.com user or group

Open an issue using the Plan Change Request template.

I got downgraded to Free/Core after my renewal, what happened?

Always go to customers.gitlab.com and check whether their namespace is linked first. If not, then ask customer to login in their Web Store account and then click on change linked namespace to do so.

It will resolve this issue.

In case above method does not solves then ask the customer to open a Support ticket.

Common Questions from Customers

Customer Question: Where can I track how many compute minutes I've used?

For individuals on GitLab.com, go to the user settings and at the bottom you see pipeline quotas where it shows the usage.

Group settings have a similar page, but they'll see Usage Quotas.

Customer Question: I bought a subscription for my personal account, how can I apply it to my group?

They just need to reassociate it with the group.

Communicate to the customer:

>>>

To do this:

- Log into: <https://customers.gitlab.com/customers/signin> using email EMAIL
- Navigate to Manage Purchases
- Select Change linked group
- Select the desired group from the This subscription is for dropdown
- Select Proceed to checkout

Note, if there are more users associated with the group (including its subgroups and nested projects) than what is included in your subscription, you will be charged for the overage of users during this last step.

>>>

How do I upgrade a customer?

1. Navigate to customer's account in Salesforce.

1. Navigate to opportunities and click into most recent purchase: should be renewal or new business, not add-on

1. From the opportunity view, navigate and click New Add On Opportunity

- This will create a new oppty for your upgrade
- Name the oppty appropriately
- From the new add-on oppty view click New Quote
- Amend subscription
- add the new SKU
- remove the old items
- Save
- Generate PDF

Billing

How does a customer pay with a credit card for a direct order?

Self-managed

Communicate to the customer

>>>

- To finalize the order, please sign and date it on the second page and return all pages back to me.

- Afterward, you will receive an automated email with the license key as well as the invoice and we will charge your credit card.

Make sure they have added a credit card on the payment methods within the customers.gitlab.com Web Store.

SaaS

Communicate to the customer

>>>

To finalize the order, please sign and date it on the second page and return all pages back to me.

Afterward, your group will be upgraded with new users and you will receive the invoice and we will charge your credit card.

>>>

Make sure that the customer has created an account, as well as a group, in order for us to apply the subscription to their group appropriately.

Payment by credit card (new customer) e-Sertifi

1. Start within quote object not from opportunity page
1. Sertifi e-sign
 - Email invite message
 - Email address, click next
 - Click related notes and attachments and select correct quote
 - Click on preview/prefill document
1. Navigate on lefthand bar to payment authorization without address
1. Move box to right of signature field
1. Save
1. Exit
1. send for signature

When they return the signed quote, no need to ping deal-desk on chatter.

OR

tag Billing Ops in chatter on the opportunity with this request

How can a customer pay via wire transfer for a direct order?

Communicate to the customer

Self-managed:

>>>

To finalize the order, please sign and date it on the second page and return all pages back to me.

Afterward, you will receive an automated email with the license key as well as the invoice for the wire transfer.

>>>

SaaS

Find the quote attached.

To finalize the order, please sign and date it on the second page and return all pages back to me.

Afterward, your group will be upgraded with new users and you will receive the invoice for the wire transfer.

>>>

Note: SaaS does not have licenses.

Customer Question: How can I initiate a refund or downgrade?

If a customer requests to downgrade the product they are currently using, we are unable to process refunds within 45 days of purchase; however, we are able to downgrade their plan.

A customer can initiate a refund themselves:

Communicate to the customer:

>>>

You can get this charged reversed by reaching out directly to AR@gitlab.com

1. Add your invoice

1. Request a refund explaining that the second charge was an error purchase

1. You must do this within 45 days of your original purchase

If you have additional questions, please let me know!

>>>

How do I initiate a refund or downgrade for a customer?

If a customer requests to downgrade the product they are currently using, we are unable to process refunds within 45 days of purchase; however, we are able to downgrade their plan. You can initiate the refund for the customer:

- Send an email to ar@gitlab.com to request a refund for the customer.
- Include all subscription information and link to SFDC account record.

Quotes

What do the terms "Sold to" and "Bill to" mean on a quote?

- Sold To email address is the one for which customers.gitlab.com account is generated and where the license key is sent to
- Bill To email address is the email address to which automatic message with the final invoice will be sent to only

Entity

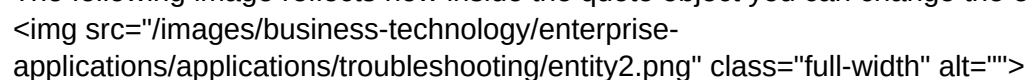
All web direct orders are GitLab Inc.

However, say you have an upcoming renewal that will be sales assisted where the customer previously purchased web direct, if the customer is located in one of the countries where we have another entity (UK, Germany, Netherlands, Australia), the renewal quote must reflect the correct entity.

E.g. Customer in UK purchased web direct, at renewal (if sales assisted) the quote should reflect GitLab Ltd. (UK entity).

This is to make sure we ask for Tax ID or VAT ID which prevents a customer from paying tax or VAT charges.

The following image reflects how inside the quote object you can change the entity to US.

The image is a screenshot of a quote object in a system, showing a field where the entity can be changed to US. The image is not visible in the provided HTML output.

Checklists for quotes

Add-on opportunity quotes

- Do I have address on account?
- Do I have address on contact?
- What is the original entity and does it match on the add-on quote?
- Is the payment correct - CC or wire transfer?

Information required for normal quote

- Full company name
- Full company address
- Sold To Contact person title, name, surname, email address
- Bill To Contact person title, name, surname, email address
- Plan
- Number of users

Copy/paste for renewals

1. Screenshot of customer admin area
1. Sold To Contact first/last name, email address, mailing address
1. Bill To Contact person first/last name, email address, mailing address
1. Plan
1. Number of users

Information required for a reseller Quote

1. Reseller Name
1. Contact Name
1. Contact Email
1. Reseller Address
1. End User Company Name
1. Contact Name
1. Contact Email

General Communications to Customers

Budget

Ask:

> Is there a budget that you are looking to stay within that we should be mindful when considering your use case?

Documentation Needed

- What do I do when I get a report of a bug or a problem with the renewal or purchase process?
- What are the instructions to process a sales assisted wire transfer?

SECTION: business-technology/enterprise-applications/entapps-services/_index.md

<link rel="stylesheet" type="text/css" href="/stylesheets/biztech.css" />

<i class="fas fa-users" id="biz-tech-icons"></i> About Us

Enterprise Application Services (EAS) provides horizontal support, operations, administration, governance, risk assessment, and compliance of Enterprise Services. Leveraging the ITIL framework, EAS is responsible for the end-to-end service and application lifecycle management, and promotes operational excellence through IT Policies & Process Governance, including Change-, Release-, and Incident Management, while ensuring operating effectiveness through Compliance Reviews.

<i class="fas fa-bullseye" id="biz-tech-icons"></i> Mission

To provide frictionless, world class, Application Support Experience for the GitLab Team members by providing easy to use Service Management tools, serving as a single point of contact for all Enterprise Applications, mitigating risk through compliance, governance, change and release management.

<i class="fas fa-users" id="biz-tech-icons"></i> Roadmap

<i class="fas fa-users" id="biz-tech-icons"></i> Our Team

Learn more about our team and roles in supporting GitLab's enterprise applications.

<i class="fas fa-bullhorn" id="biz-tech-icons"></i> Our Services

Service Operations

- Application LifeCycle Management
- Systems Administration
- Budget & Vendor Management
- Application Rationalization
- New Software Assessment and Approvals
- Change & Release Management

IT Compliance (SOX, GRC)

- ITGC
- External & Internal Audit Support
- User Access REview
- Separation of Duty

<i class="fas fa-users" id="biz-tech-icons"></i> How We Operate

- Agile Methodology as Operational Standard.
- Engagement Model
- Major Incident Management
- Change Management (Coming Soon)

- Release Management (Coming Soon)
- New Application SOX Compliance Review (Coming Soon)

SECTION: business-technology/enterprise-applications/finance-ops/index.md

<link rel="stylesheet" type="text/css" href="/stylesheets/biztech.css" />

<i class="fas fa-users" id="biz-tech-icons"></i> About Us

The Enterprise Applications, G&A Team is responsible for administering and expanding GitLab's financial systems to meet the demands of scalability, automation, and regulatory compliance. Through proactive monitoring, alerting, and continuous iteration, we ensure the stability and reliability of GitLab's internal applications ecosystem.

Our suite of financial tools includes:

- NetSuite (ERP)
- Coupa (Procurement)
- Zip (Vendor Management)
- Navan Travel & Expense (Travel Management)
- Adaptive Planning (Financial Planning)

Additionally, we oversee IT Audit and Compliance processes, ensuring adherence to IT General Controls (ITGC) to meet SOX and other regulatory standards.

<i class="fas fa-bullseye" id="biz-tech-icons"></i> Mission

Our mission is to empower GitLab's business processes by delivering an integrated and efficient application landscape. We strive to:

- Enable seamless workflows: Create a cohesive ecosystem where enterprise applications work together, delivering a superior user experience for business partners.
- Ensure data integrity and security: Maintain robust processes to protect sensitive data and ensure consistent accuracy across systems.
- Drive efficiency through automation: Simplify workflows and maximize automation by leveraging industry best practices.
- Focus on core competencies: Adopt and extend third-party solutions where appropriate, allowing GitLab's engineering resources to focus on core product innovations.
- Maintain audit and compliance standards: Safeguard customer and business data to meet key audit requirements, including SOX and SOC compliance.

<i class="fas fa-users" id="biz-tech-icons"></i> Our Team (Org Chart)

Our Team (Org Chart)

<i class="fas fa-bullhorn" id="biz-tech-icons"></i> Our Services

System Evaluations & Implementations

We collaborate with the Program Management Office (PMO) to evaluate, implement, or configure new applications or major updates. By gathering and validating user requirements, we ensure applications align with business needs, driving automation and efficiency.

Finance Systems Administration

Our team ensures smooth operations of GitLab's financial systems, including:

- User provisioning and offboarding
- Chart of Accounts (COA) updates
- Financial report creation and maintenance
- Release upgrade testing
- Daily system upkeep to ensure operational continuity

Non-Project Operational Enhancements

We prioritize and address smaller-scale, non-project tasks to enhance system functionality. These include:

- Workflow updates and custom field enhancements
- Enabling quick features
- Policy updates for expenses and other configurations

How We Operate

Request Submission Process

- Projects: For initiatives requiring several weeks, cross-functional impact, or project management support, contact the PMO team to include the request in the project roadmap.
- Operational Enhancements: Submit smaller system changes through the CM: Configuration Change template linked [here](#). Requests are evaluated and prioritized using the ~BT:Backlog label [here](#).

Milestone Planning

Our team works on a bi-weekly milestone schedule (Wednesday-Tuesday).

- Open milestones are tracked [here](#).
- At the close of each milestone, unresolved issues roll over to the next milestone. Additionally, new issues in the backlog will be evaluated to determine the resources required and feasibility of starting it in the new milestone.

Note: Requests tied to operational continuity (KTLO) or financial impact are prioritized over non-critical enhancements.

Our Commitment

Through proactive support, strategic improvements, and adherence to compliance standards, the Enterprise Applications, G&A team ensures that GitLab's financial systems not only meet today's demands but are also prepared to support future growth. By maintaining robust processes and

fostering collaboration across departments, we empower GitLab to operate with agility and excellence.

SECTION: business-technology/enterprise-applications/guides/_index.md

<link rel="stylesheet" type="text/css" href="/stylesheets/biztech.css" />

One of the team's initiatives has been to conduct short training sessions on different topics within the Enterprise Applications domain.

Why we do this:

- Increase the base level of system and process knowledge across the team
- Provide a more homogeneous level of support to our business partners on key topics
- Allow team members to self-service as much as possible and reduce disruptions for small tasks

Finance System Guides

- How to create custom fields in Netsuite
- Coupa End User Guide
- Zip End User Guide
- Product Catalog Guide
- Navan Expense End User Guide

SECTION: business-technology/enterprise-applications/guides/coupa-supplier.md

<link rel="stylesheet" type="text/css" href="/stylesheets/biztech.css" />

How to create a new supplier in Coupa

TBD

SECTION: business-technology/enterprise-applications/guides/ns-custom-field.md

<link rel="stylesheet" type="text/css" href="/stylesheets/biztech.css" />

How to create custom fields in NetSuite

Recording (private due to finance systems information): <https://youtu.be/7QEw0-E4dTQ>

1. Navigate to Customization > Lists Records & Fields > Transaction Body Fields
1. Click New
1. Set fields
 1. Label

1. ID The id that's used when interacting through the API
1. Owner
1. Description
1. Type
1. Store Value ticked
1. Applies to section, select the entities where this field will be available
1. Display Tab (controls where the field would show in the UI)
 1. Main SubTab
 1. Insert before (pick an existing field)
 1. Default access level View
1. You can then navigate to the entity and see if the new field is displaying

SECTION: business-technology/enterprise-applications/guides/product-catalog.md

<link rel="stylesheet" type="text/css" href="/stylesheets/biztech.css" />

Change Management and SDLC Process

For SOX/audit purposes, all changes to the Zuora Billing product catalog must be properly tested, adhering to Business Technology Change Management policies and the Software Development Lifecycle Process for Finance Systems.

How to request the creation or modification of a SKU

Before submitting a SKU request issue, please read and complete all steps of the Product Launch Process. This will help ensure that the product launch is done efficiently and effectively. A business sponsor or program manager should be assigned and have the business requirements clearly defined before creating an issue for SKU c